

## 5th Place Solution

|                 |          |
|-----------------|----------|
| <b>Rank</b>     | 5        |
| <b>Team</b>     | Dieter   |
| <b>Author</b>   | Dieter   |
| <b>Topic ID</b> | 679360   |
| <b>Version</b>  | Original |

Source: <https://www.kaggle.com/competitions/vesuvius-challenge-surface-detection/discussion/679360>

Retrieved with Kaggle CLI on 2026-07-07.

First, let me thank the Vesuvius Challenge team and Kaggle for organizing this fascinating competition.

## TLDR

---

My solution is an ensemble of UNets mainly based on a custom SEResNeXt152 encoder with an Attention UNet decoder. All models are trained to regress a signed distance field (SDF) rather than a binary mask. I am using a novel(?) SDF L1 + SDF mass loss which is the sdf equivalent of BCE + Dice. SDF predictions are averaged across checkpoints and up to 8 TTA flips. At the end all is binarized at an SDF threshold of 0.3, and then refined with an iterative persistence-homology-based tunnel-filling post-processing pipeline. Although training on 160x160x160 crops, I inferred on full 320x320x320 to prevent any artifacts from a sliding window approach.

## Cross-Validation

---

I used 4-fold cross-validation. During training I tracked three metrics locally - Surface Dice, VOI accelerated by GPU on original size, and the topology score on 10x downsampled size on CPU as the bettimatching algorithm is sequential. Neither local CV nor public LB correlated well with private LB, hence my selection of final submission went poorly.

## Training Routine

---

All models are trained on  $160^3$  ROI patches randomly cropped from the npy saved  $320^3$  training volumes. SDF targets are calculated on the fly. Training uses the Adam optimizer with learning rate  $1e-3$ , cosine annealing schedule. Mixed-precision training with bfloat16 is used throughout, and gradient checkpointing is enabled for the larger SEResNeXt152 models to fit within GPU memory. Weight clamping to the fp16-safe range is applied at checkpointing time to ensure clean float16 inference later. Batch size is 16.

Data augmentation consists of random 3D flips along each spatial axis ( $p=0.5$  each) and random 90-degree rotations in the (axis-1, axis-2) plane ( $p=0.5$ ). CutMix ( $\beta=1.0$ ) was used in earlier model families but disabled for the final SEResNeXt152 runs, where it did not improve scores.

## Loss Function: SDF L1 and "mass"

---

The key design decision was to train models to regress a signed distance field rather than a binary segmentation mask. The SDF target is computed from the binary label via Euclidean distance transforms on the fly on GPU:

```
fg_edt = distance_transform_edt(foreground_mask)
bg_edt = distance_transform_edt(~foreground_mask)
sdf = bg_edt - fg_edt          # negative inside, positive outside, zero at boundary
sdf = clamp(sdf, -100, 5)
```

The loss has two components with equal weight:

**Weighted SDF L1 loss (weight 0.5):** A voxel-wise L1 loss between predicted and target SDF, weighted by a Gaussian bell-curve matrix that concentrates supervision near the surface boundary:

$$W(\text{sdf}) = 1 + w_0 * \exp(-((\text{sdf} + \text{core\_radius})^2) / (2 * \text{sigma}^2))$$

with  $w_0 = 8$ ,  $\text{sigma} = 4$ ,  $\text{core\_radius} = 5$ . This gives a peak weight of 9.0 at  $\text{sdf} = -5$  (inside the surface at depth 5 voxels), plateauing at 9.0 for deeper interior, and decaying to  $\sim 1.0$  far from the surface. The effect is that the model receives a strong gradient signal near the surface while distant background voxels contribute minimally.

**SDF-based Dice loss (weight 0.5):** Instead of thresholding to binary, a continuous "mass" is derived via  $\text{ReLU}(-\text{sdf})$  for both prediction and target, and a soft Dice coefficient is computed from these masses. This provides a global volumetric overlap signal that complements the voxel-wise L1.

The following figure illustrates the three quantities at a central slice ( $\text{axis}_0 = 160$ ) of a training volume:

Left: original binary target (0 = background, 1 = foreground, 2 = ignore). Center: SDF target (negative inside surface, positive outside; black contour at  $\text{SDF} = 0$ ). Right: Gaussian weight matrix (higher weights near and inside the surface boundary).

This loss combined with gaussian weighting solves a lot of the key challenges of this competition in an elegant and efficient way: Sheet separation, sheet continuity, boundary sharpness, precise skeletons etc.

## Model Architectures

I explored a wide range of 3D architectures. Instead of relying on frameworks like nnUnet or medcaI I asked an agent to recode relevant model families from scratch. This gives much more control. For the keras based SEResnext model from medcaI it was also 60% (!) faster for training. The final ensemble draws from three architecture lineages:

**SEResNeXt152 + Attention UNet** -- the main workhorse and strongest single model. A custom pure-PyTorch 3D SE-ResNeXt152 encoder (layers [3, 8, 36, 3], groups=32, width\_per\_group=8, SE reduction=16, DropPath rate 0.3) paired with an Attention UNet decoder (channels [256, 128, 64, 32, 16]). Attention gates on skip connections let the decoder focus on relevant encoder features. Gradient checkpointing is essential here due to the 36-block third stage. One variant using `seresnext152_48x8d` (48 groups, width 8) for additional capacity.

**ResNet152 + skip maxpool** -- Expensive UNet with a ResNet152 encoder, custom decoder channel configuration, and deep supervision during training (auxiliary losses at intermediate decoder stages). I skip one early maxpool so resolution through the model is higher. At inference, only the final head is used.

**ResNet152 + UNet with SDF loss heavy augs and external data** -- earlier ResNet152-based UNets trained with the SDF + Dice loss. One model extends the training set with additional scrolls. I used an additional binary input channel to give the model pixelwise information that this data is external (and hence labels are "bad") These models provide useful diversity to the ensemble despite being individually weaker than SEResNeXt152.

I also experimented with SwinUNETR, UNETR++, MedNeXt, SegFormer3D, ConvNeXt V2, U-Mamba, and SEResNeXt200, but none surpassed the SEResNeXt152 + AttUNet architecture on local validation. Some topology-aware loss variants (persistence diagram matching loss, Euler characteristic loss) were explored. One PD-matching variant is included in the final ensemble but its finetuned only for a few epochs from another model because training was very slow.

# Ensemble

---

Predictions are combined by simple averaging of raw SDF logits across all checkpoints and TTA augmentations per sample. For the larger SEResNeXt152 models, TTA is reduced to 2 augmentations (identity + triple-flip) to stay within the 9-hour Kaggle kernel runtime; smaller models use the full 8-flip TTA (3 single-axis flips + 3 dual-axis + 1 triple-axis + identity).

NaN safety is critical: the SEResNeXt152 models are trained in bfloat16 but inference runs in float16. A safety conversion routine clamps all weights to `[-65504, 65504]` and registers forward hooks that replace any NaN/Inf activations.

# Inference

---

All inference is done on full 320x320x320 image! No strided slices, as those create artifacts that are very harmful to topology. Two GPUs are used in parallel via data sharding: even-indexed test samples go to GPU 0, odd-indexed to GPU 1. Each GPU processes all models sequentially for its shard and offload predictions to disk to prevent OOM.

# Post-Processing

---

Post-processing proved essential for the topology score, which heavily penalizes tunnels (H1 topological features) in the predicted surface. My pipeline:

**Step 1 -- Binarization:** Threshold the averaged SDF logits at 0.3 (voxels with  $\text{SDF} < 0.3$  become foreground).

**Step 2 -- Dust removal:** Remove small connected components ( $< 50,000$  voxels, 6-connectivity), preserving components that touch at least 3 volume boundaries regardless of size.

**Step 3 -- Iterative H1 tunnel filling** (13 iterations):

1. **SDF filtration:** Overwrite foreground voxels in the raw SDF to `threshold - 1.0`, creating a cubical filtration where the surface sits at the threshold.
2. **Persistence barcode computation:** Split the volume into  $2 \times 2 \times 2 = 8$  octant tiles, compute persistence homology on each tile using a custom C++ module (`barcode3d_fast_v3`) which was derived from betti matching library. This identifies H1 features (tunnels) with birth/death values and coordinates.
3. **Straddling filter:** Keep only H1 features where `birth < sdf_threshold < death`, i.e., tunnels that cross the binarization surface.
4. **Adaptive radius:** For each tunnel's death coordinate, estimate the tunnel width from the background EDT and set the fill radius to `clip(edt_width + 1.0, 3.5, 7.0)`.
5. **Bridge detection:** Simulate filling a ball at each death coordinate and check whether it would merge separate connected components. Skip coordinates that would create bridges.
6. **Ball filling:** Fill spherical balls at the remaining death coordinates. On the final iteration, also apply morphological hole filling.
7. **Component count guard:** If filling reduced the number of connected components (accidental merge), revert to the pre-fill state.

## Key Insights

- **SDF regression > binary classification** for this metric suite. The SDF naturally encodes distance-to-surface information, and thresholding the SDF at different values during validation gives a smooth trade-off between Surface Dice and topology metrics. Binary cross-entropy models consistently scored lower on the topology component.
- **Gaussian weighting** focuses the loss on the skeleton region, but still extends to sheet boundary. Background is heavily downweighted.
- **Iterative tunnel filling** with bridge detection is the single most impactful post-processing step. Going from 0 to 13 iterations improved the topology score by ~0.08 on local validation, with diminishing returns beyond 11-13 iterations.

## Supplemental Q&A

Innat · 2026-03-01T05:43:40.037000

@christofhenkel Amazing experiment. I really enjoyed reading your write-up. This approach is quite different from what others have explored. Congratulations on the solo win!

Have you open-sourced your training and inference code? I would love to explore it for learning purposes.

For the keras based SEResnext model from medcai it was also 60% (!) faster for training.

That is a huge improvement. Could you please elaborate on this in more detail? Did you run controlled benchmarks or compare training times against publicly available notebooks? It would really help me understand whether there is still room to optimize the medcai implementation.

Dieter · 2026-03-01T06:29:14.963000

I did not run a controlled benchmark, but I can share the before and after. I am really happy that medcai exist, it has a perfect repository structure and documentation. Only problem is keras. Its important to note that I used keras with pytorch backend, and I guess that just does not work well. I got a memory leak with that model and used jit trace to fix it, so first I used this

```
import os
os.environ["KERAS_BACKEND"] = 'torch'
from medcai.models import AttentionUNet

model = AttentionUNet(encoder_name='seresnext50', input_shape=(160,160,160,1),
    classifier_activation=None, num_classes=1)
dummy_input = torch.randn(size=[1] + cfg.roi_size + [1])
traced_model = torch.jit.trace(model, dummy_input, strict=False, check_trace=False)
self.backbone = traced_model
```

The reimplementation is attached.

Giorgio Angelotti · 2026-03-01T09:03:17.683000

This solution is really brilliant and I like seeing architectures coming out of the nnUNet framework! I find the formulation as an SDF regression really elegant. When we launched this challenge we thought that some

solutions could be inspirational also for another task we are working on: ink detection in the scrolls. I can see this solution very easily readapted for ink detection as well!

I have two questions

1. The part of the loss with mass from the SDF and the soft Dice made me think of [a paper I read](#) a while ago where the authors said that in tasks with uncertainty in the labels, optimization of the soft dice can lead to biased estimates of the volume of the region to segment, while the cross entropy leads to worse Dice score but unbiased volume estimates. Maybe an overestimation of the volume can lead to unwanted merges, irregardless of the better Dice? You say that\* Binary cross-entropy models consistently scored lower on the topology component\*, but I am curious to know whether you tested also a BCE variant of the loss on a probability derived from the SDF, e.g.  $\text{sigmoid}(-\text{sdf}/T)$  (with T some temperature)?
2. Within the architectures that you are using, the ResNe(X)ts have a lot of layers. Do you feel that this higher capacity is really needed for this task? I am also interested in the skip of the first maxpool to try to preserve finer structure. I think this can be even more important for ink!

Thank you!

**Cody\_Null · 2026-03-01T21:11:34.050000**

Glad to see another gold with an interesting approach. I originally wanted to try out SEResNeXt152 type models but we had so much success with the nnunet structure and some of the custom models that we started with that I never even tried it. Very jealous of being able to infer on full 320,320,320 with 8xTTA. As our approach had so many steps by the time we got to post processing we only had about an hour for inference time. I would be curious to see how much impact that inferencing resolution had on your cv/lb? Another question I had was our model preformed worse when averaging on raw logits instead of the probabilities, despite the logits being the more logical approach, so I would be curious if you tried that. Love the post processing ideas as well.

I really thought after seeing everyones solutions that we would have a solid pipeline and just missed with the post processing and that slid us down to 10th but so far all the post processing from other top competitors actually just lowers our score.

Congrats on your placement!

**Dieter · 2026-03-02T07:31:18.530000**

1. The paper author claim "systematic under or overestimation of the predicted volume". I guess this can be true. However note that I am using a binarization threshold of 0.3 which systematically predicts a slightly "larger" volume than the model predicted (higher sdf means further away from sheet center). This threshold is optimized via grid-search on CV and incoorporate trade-off of the 3 metrics used. In short, you can control merges with the threshold. I also tried more targeted prevention of merges via adding an sdf weight at voronoi lines, but it had no effect, because model was already really good at separation. I think reason is that sdf-approach results in really good skeletons. I tested probability derived from SDF, especially since this then can be used for frangi filter etc, but it performed worse and its tricky to tune T.
2. Yes higher capacity has significant impact. I tend to start with very simple and fast models to optimize the number of experiments/ time for a competition and only increase capacity if really needed.

Dieter · 2026-03-02T07:57:14.437000

Very jealous of being able to infer on full 320,320,320 with 8xTTA

Reading your inference kernel it seems you used an overlap of (0.5,0.5,0.5) with a  $160^3$  window. Thats 27 patches for a  $320^3$  image and 3x the amount of voxels put through the model. So even with TTA8 my inference is 1.5x cheaper without creating border artifacts.

My model simply has no probabilities. It predicts sdf values (ranging from -10 to 5) and then binary mask, by thresholding at 0.3. As [@giorgioangelotti](#) explained you can create artificial probabilities by introducing a temperature and do  $\text{sigmoid}(-\text{sdf}/T)$ , but T is another hyperparameter and probs will be on a slightly different scale than those from BCE models. You can (probably) ensemble my model with other models by a voxel rank transform like we did in our [winning solution to the CryoET competition](#) section Ensembling

Bartley · 2026-03-02T19:02:17.843000

Congrats on 50 golds [@christofhenkel](#), great achievement!

Do you design your architecture/training pipeline any differently when the inference patch size differs from the training patch size?

I often struggle to match the performance achieved when using the same patch size during training and inference (in this competition and past 3D competitions).

Dieter · 2026-03-02T19:51:34.867000

Its the first time I used much larger patch size for inference. Learned the trick from [@bloodaxe](#) in CryoET comp. I think the metric in this competition favors not doing slided window inference (SWI) a bit because border artifacts and inhomogenous amount of voxel predictions (think about that with SWI some voxels are predicted more often than others) hurt the topo score. And what I saw here is that SWI hurt the score more than inconsistencies created by the patch size mismatch. E.g. I normalized each patch by mean/ std and thats inconsistent for different patch sizes.